

Approximate Multiply-Accumulate (MAC) Unit for Energy-Efficient Edge AI Inference

Adita Faruq

BSc in EEE, University of Dhaka

Unive Mentorship Program – Summer 2026, Hands-On Project Proposal

Application Domain

Power & Energy Management

Edge AI devices such as, wearables, IoT sensors run neural network inference under strict power and area budgets. The core operation in this inference, multiply-accumulate dominates both the power consumption and chip area of these devices which makes it a direct power-management problem at the hardware level.

Objective

To design, implement and physically build (RTL $\rightarrow$ GDSII) two versions of a Multiply-Accumulate unit, one exact and one approximate, and carry both through the full OpenROAD-flow-scripts physical design flow on the open IHP SG13G2 (130 nm) PDK in order to quantify the power/area savings against the accuracy cost of approximation.

Problem Statement

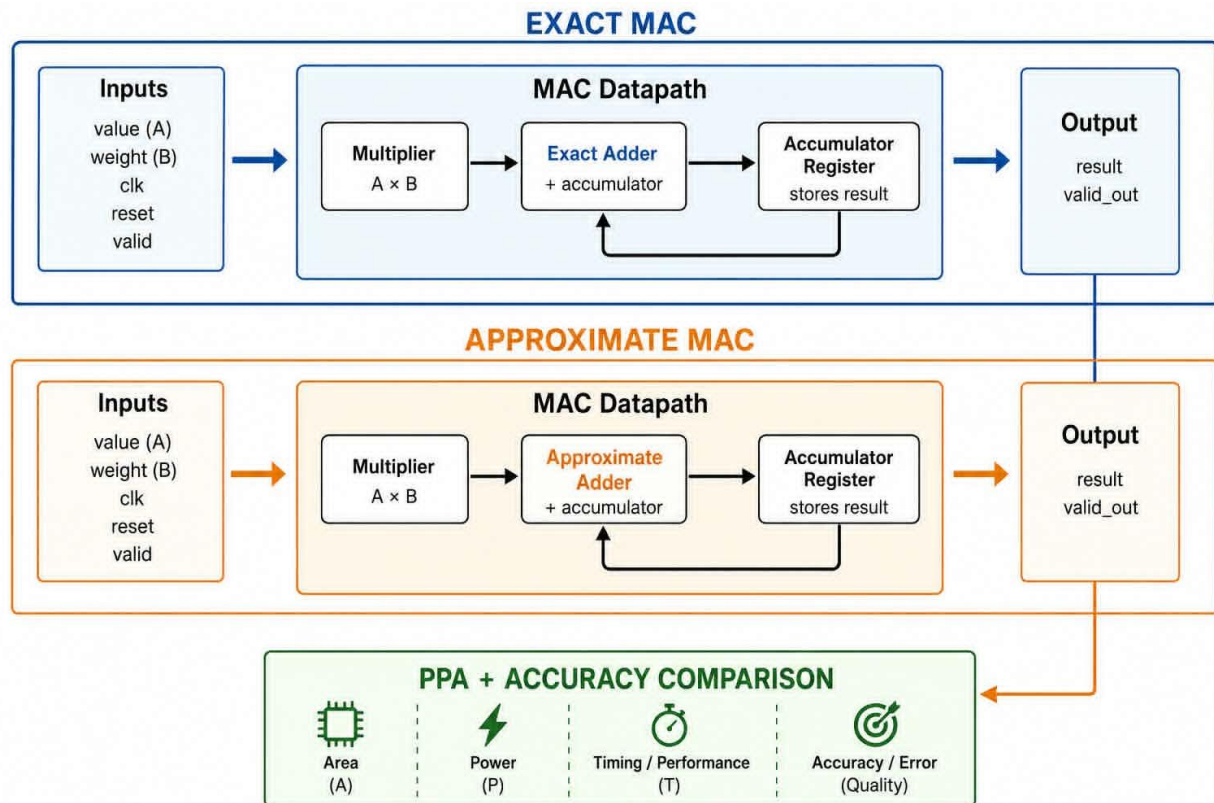
Exact multipliers and adders in a MAC unit consume significant power and area because of full carry propagation and complete partial-product summation. Neural network inference is inherently error-tolerant - small numerical errors in intermediate computations rarely change the final output. This project asks: How much power and area can be saved by introducing controlled approximation into a MAC unit, and what accuracy cost does this approximation introduce? Answering this with real, physically-implemented numbers gives a concrete, hardware-verified answer for edge-AI chip design decisions.

Project Overview

The design takes two N-bit fixed-point inputs (a value and a weight) each cycle, multiplies them, and adds the result to a running accumulator register. Two versions are built from the same top-level structure:

- **Exact MAC:** standard multiplier + full-adder-based accumulate stage.
- **Approximate MAC:** uses the same overall architecture, but incorporates a selected approximate adder in the accumulation stage to reduce hardware complexity and switching activity at the cost of controlled numerical error. Both variants are pushed through the same OpenROAD flow independently, and their synthesis/PnR reports and output accuracy (measured against a set of test input vectors) are compared directly.

Block Diagram



Key Features

- Fully synchronous, single flat RTL module with no sub-modules, no memory macros.
- Parameterizable bit-width for inputs and accumulator.
- Two variants (exact / approximate) built from a shared structure for direct comparison.
- Verified with a self-checking testbench comparing approximate output error against exact output across a range of test vectors.

Design Specifications

Parameter	Value
Technology	IHP SG13G2 (130 nm, open PDK)
Language	Verilog HDL (synthesizable)
Physical Design Flow	OpenROAD-flow-scripts (ORFS)
Target Clock Period	10.0 ns (100 MHz), SG13G2 starting point
Data Inputs	$2 \times N$ -bit fixed-point (value, weight)
Control I/O	clk, reset, valid
Status Outputs	result, valid_out
Memory Macros	None (flip-flops + combinational logic only)

Expected Deliverables

- RTL for both variants, SDC and ORFS config.mk in a GitHub repository.
- Screenshots of each design stage (floorplan, placement, CTS, routing, final layout) for both variants.
- Reports on area, power, timing (WNS/TNS), utilization, cell count, and clock tree summary - compared side by side.
- Final GDSII layouts and gate-level netlists for both variants.
- A technical report covering methodology, implementation flow, PPA-vs-accuracy results, challenges, and conclusions.